

Low Latency Live Streaming Implementation in DASH and HLS

Abdelhak Bentaleb
NUS, Singapore
bentaleb@comp.nus.edu.sg

Zhengdao Zhan
NUS, Singapore
zhengdao@u.nus.edu

Farzad Tashtarian
AAU, Austria
farzad.tashtarian@aau.at

May Lim
NUS, Singapore
maylim@comp.nus.edu.sg

Saad Harous
UoS, UAE
harous@sharjah.ac.ae

Christian Timmerer
AAU, Austria
christian.timmerer@aau.at

Hermann Hellwagner
AAU, Austria
hermann.hellwagner@aau.at

Roger Zimmermann
NUS, Singapore
rogerz@comp.nus.edu.sg

ABSTRACT

Low latency live streaming over HTTP using *Dynamic Adaptive Streaming over HTTP* (LL-DASH) and *HTTP Live Streaming* (LL-HLS) has emerged as a new way to deliver live content with an respectable video quality and short end-to-end latency. Satisfying these requirements while maintaining viewer experience in practice is challenging, and adopting conventional adaptive bitrate (ABR) schemes directly to do so will not work. Therefore, recent solutions including LoL⁺, L2A, Stallion, and Llama re-think conventional ABR schemes to support low-latency scenarios. These solutions have been integrated with dash.js [9] that supports LL-DASH. However, their performance in LL-HLS remains in question. To bridge this gap, we implement and integrate existing LL-DASH ABR schemes in the hls.js video player [18] which supports LL-HLS. Moreover, a series of real-world trace-driven experiments have been conducted to check their efficiency under various network conditions including a comparison with results achieved for LL-DASH in dash.js. Our version of hls.js is publicly available at [3] and a demo at [4].

CCS CONCEPTS

• Information systems → Multimedia streaming.

KEYWORDS

Low Latency Streaming, ABR, LL-DASH, LL-HLS, hls.js, dash.js.

ACM Reference Format:

Abdelhak Bentaleb, Zhengdao Zhan, Farzad Tashtarian, May Lim, Saad Harous, Christian Timmerer, Hermann Hellwagner and Roger Zimmermann. 2022. Low Latency Live Streaming Implementation in DASH and HLS. In *Proceedings of the 30th ACM International Conference on Multimedia (MM '22)*, Oct. 10–14, 2022, Lisboa, Portugal. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3503161.3548544>

1 INTRODUCTION

HTTP Adaptive Streaming (HAS) has become the dominant media delivery system allowing various industries to deliver their contents and engage with customers efficiently. There are two mainstream HAS delivery systems in the market: MPEG Dynamic Adaptive Streaming over HTTP (MPEG DASH) [13] and HTTP Live Streaming (Apple HLS) [17]. Both HAS technologies share a common design principle. On the one hand, the server (or content delivery network (CDN)) provides segmented video contents at multiple bitrate levels allowing adaptation to different network conditions. On the other hand, a client uses an Adaptive Bit Rate

(ABR) scheme to decide the suitable bitrate level to download from the server (or CDN) according to the stream information given in the DASH Media Presentation Description (MPD) or HLS playlist.

In the past decade, there has been a tremendous shift from traditional cable TV to Internet TV because of technology advances in online video streaming over the Internet. Video services such as YouTube, Facebook, Twitch, Netflix and others occupy a huge part of today's Internet ecosystem. Significant growth in video traffic poses many challenges for live video streaming providers and might limit their ability to deliver to the viewer an assured quality of experience (QoE). Therefore, *low latency live* (LLL) streaming has received tremendous attention by both the research community and the industry. To achieve the low latency objective, DASH and HLS have been extended to support LLL streaming scenarios.

Low latency DASH (LL-DASH) [10] was first introduced in 2018, followed by low latency HLS (LL-HLS) [5] in 2020. Both extensions share common design principles following the Common Media Application Format (CMAF; ISO/IEC 23000-19) specification [12]. CMAF allows each media segment to be encoded and delivered in small non-overlapping chunks (also called parts). The client starts downloading each chunk of a segment once it is prepared and available at the origin server. The chunks of a segment will be sent to the client one by one, and the client can start rendering a segment right after its first chunk is received. Therefore, the glass-to-glass latency, *i.e.*, the time lag between video capture and the actual playback time at the client, is decoupled from the segment duration and reduced to a few seconds (*i.e.*, less than five seconds). LL-DASH differs from LL-HLS on how to deliver chunks to the client where the former uses HTTP/1.1 chunked transfer encoding (CTE) while the latter uses HTTP/2. Although LL-DASH and LL-HLS are a step forward in LLL streaming, adopting one of them directly is not sufficient to reduce latency. In addition, auxiliary tasks such as performing ABR decisions and estimating bandwidth become more challenging. Furthermore, existing conventional ABR schemes cannot be used directly in low-latency scenarios.

Recently, interesting solutions including LoL⁺ [6], L2A [14], Stallion [11], and Llama [16] have been developed to address these issues leveraging LL-DASH. These solutions have shown the best performance in LL-DASH. However, their adaptation to and performance in LL-HLS is the main focus of this paper. The contributions of this paper are two-fold: (1) We provide a description of these ABR schemes including their integration and practical implementation in the open source video player hls.js. (2) We validate the efficiency of different integrated ABR schemes over real-world network conditions using realistic trace-driven emulations. We contribute our code to the hls.js main repository [18]. The code is publicly available at [3] and demonstrated at [4].



This work is licensed under a Creative Commons Attribution International 4.0 License.

MM '22, October 10–14, 2022, Lisboa, Portugal
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9203-7/22/10.
<https://doi.org/10.1145/3503161.3548544>

2 LL-DASH SOLUTIONS

We briefly describe existing LL-DASH solutions that are implemented in the dash.js [9] media player.

Learn2Adapt (L2A). L2A [14] utilizes online convex optimization to formulate the bitrate decision problem. The main objective of L2A is to minimize the overall latency in the LLL session, while achieving a high video bitrate and therefore acceptable QoE. Moreover, it provides improved robustness against fast changes in the critical characteristics, including throughput and buffer level. It solves the optimization problem using an online learning approach that does not require parameter tuning, channel model assumptions, throughput estimation or application-specific adjustments.

LL Adaptive Media Algorithm (Llama). Llama [16] is a recently introduced ABR scheme that divides the throughput measurement module of the player into two parts. The first part considers the last measured throughput of the last downloaded segment to perform bitrate reduction decisions while the second part takes advantage of the harmonic mean of the last twenty downloaded segments to contribute to video quality up-switch. This design enables more active reactions when facing worsening network conditions, while it prevents a potential rapid response to temporary fluctuations with the smooth estimation brought by the harmonic mean.

Standard LL Video Control (Stallion). Stallion [11] is a throughput-based ABR scheme for LLL streaming. The main idea behind Stallion is to use a segment-based sliding window algorithm of the last ten downloaded segments to measure the mean and standard deviation (std) of both the throughput and the latency, which are then utilized to perform ABR decisions. Therefore, using the std along with the mean provides a safer prediction for available bandwidth and thus reduces the likelihood of rebuffering.

Low On Latency Plus (LoL⁺). LoL⁺ [6] re-visits and extends several important components at the client-side in ABR streaming to support LLL scenarios. LoL⁺ includes five essential modules. (1) *Throughput Measurement*: LoL⁺ implements a novel algorithm to address the issue of idle periods [8] in LLL streaming for accurate measurements. (2) *Weight Selection*: LoL⁺ adapts a Self Organizing Map (SOM) [15], an unsupervised machine learning technique, for bitrate selection. SOM is well-known to be very sensitive to the initial weight values. For this reason, the module implements a weight selection algorithm that is invoked at every segment download step to pick suitable weights associated with the set of considered SOM features. (3) *QoE Evaluation*: It implements a QoE function that combines five metrics—selected bitrate, bitrate switches, rebuffering duration, latency, and playback speed—helping the bitrate selection module in choosing a suitable bitrate for the next segment to be downloaded. (4) *Playback Speed Control*: It dynamically adjusts the playback speed (normal, speed-up, or speed-down) in order to maintain the latency within its target range while avoiding the risk of rebuffering. (5) *Bitrate Selection*: It implements the LoL⁺ ABR scheme that is triggered at each segment download. It formulates the bitrate selection problem as an unsupervised classification, leveraging the SOM [15] technique. For every segment download, the SOM model takes a quadruplet of features (throughput, rebuffering duration, latency, and bitrate switches) as input and then finds and selects the best matching unit (BMU) that is associated with the best bitrate.

3 LL-DASH SOLUTIONS OVER HLS.JS (LL-HLS)

3.1 hls.js Overview

hls.js [18] is an open-source library that implements an HLS player. It relies on the HTML Video Element and the Media Source Extensions to play HLS streams directly within a browser. It works by transmuxing MPEG-2 Transport Streams and AAC/MP3 streams into ISO BMFF (MP4) fragments. It is written in ECMAScript6 (*.js) and TypeScript (*.ts). Figure 1 highlights the essential modules:

- **Controllers.** This module implements a set of controllers that is required by the video player to perform various tasks. It comprises: (i) an ABR Controller (`abr-controller.ts`) that implements the ABR logic for bitrate selection; (ii) a Buffer Controller (`buffer-controller.ts`) responsible for monitoring the playback buffer occupancy level to avoid rebuffering events or buffer overflow; (iii) a Stream Controller (`stream-controller.ts`) handling all functionalities (e.g., load, start, stop stream) related to HLS streams as well as managing other existing controllers; (iv) a CMCD Controller (`cmcd-controller.ts`) implementing various functionalities of common media client data defined in the specification [2]; and (v) a Latency Controller (`latency-controller.ts`) responsible for maintaining latency within its target range and also changing the playback speed rate if required.
- **Loader.** This module consists of: (i) Fragment Loader (`fragment-loader.ts`) to request and download a segment from a server; (ii) Key Loader (`key-loader.ts`) to load and decrypt the key associated with the HLS stream for digital rights management (DRM); and (iii) Playlist Loader (`playlist-loader.ts`) to handle media manifest/playlist loading tasks.
- **Utils.** This module implements a set of classes required by various other modules such as: Fetch Loader (`fetch-loader.ts`) and XHR Loader (`xhr-loader.ts`) for requesting and downloading segments, EWMA Bandwidth Estimator (`ewma-bandwidth-estimator.ts`) for bandwidth estimation using an exponentially weighted moving average smoothing function, and MP4 Tools (`mp4-tools.ts`) to parse MP4 data.
- **Demux.** This module is responsible for reading the multi-part (audio, video, and subtitles) HLS stream and saving each part as a separate stream. Its main class is Transmuxer (`transmuxer.ts`) which uses TS Demuxer (`tsdemuxer.ts`), AAC Demuxer (`aacdemuxer.ts`) and MP3 Demuxer (`mp3demuxer.ts`) for audio, and MP4 Demuxer (`mp4demuxer.ts`) for video.
- **Remux.** This module is responsible to take the video and audio streams from one container and put them into a new MP4 container using the MP4 Remuxer (`mp4-remuxer.ts`) with the help of MP4 Generator (`mp4-generator.ts`; to generate MP4 Box) and Pass Through Remuxer (`passthrough-remuxer.ts`).
- **Crypt.** This module mainly uses Decrypter (`decrypter.ts`), AES Crypto (`aescrypto.ts`), AES Decryptor (`aesdecryptor.ts`), and FAST AES Key (`fast-aes-key.ts`) classes to facilitate DRM operations and decrypt HLS streams.

3.2 LLL Solutions Integration with hls.js

Our modifications are mainly performed in three modules: *Controllers*, *Loaders*, and *Utils*. Specifically, we added nine new

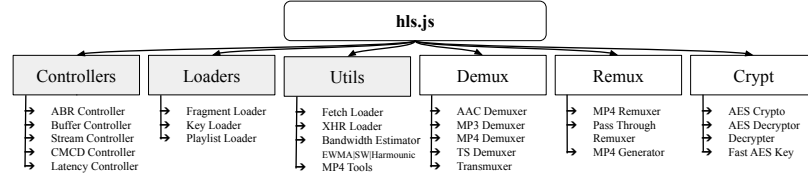


Figure 1: Overview of hls.js modules. The modules in gray are where our modifications have been performed.

Table 1: Our integration of LLL streaming solutions in hls.js.

Module	Class	State	Code Lines
Controllers	L2ARule.js	New	574
	LlamaRule.ts	New	107
	StallionRule.ts	New	129
	LoLPRule.js	New	571
	LoLPWeightSelector.ts	New	207
	LoLPQoeEvaluator.ts	New	208
	LoLPQoeInfo.ts	New	51
	abr-controller.ts	Modified	377
Utils	sw-bandwidth-estimator.ts	New	36
	hm-bandwidth-estimator.ts	New	36
	fetch-loader.ts	Modified	36
Loaders	fragment-loader.ts	Modified	3
	loader-state.ts	Modified	4
	loader.ts	Modified	8

classes and modified five classes. Table 1 summarizes the newly added and modified classes in hls.js (v1.0.0).

Each added ABR scheme is enabled in `abr-controller.ts`. In addition, we modified (a) `abr-controller.ts` to add the LoL⁺ playback speed control module, and (b) `fetch-loader.ts`, `loader.ts`, `loader-state.ts`, and `fragment-loader.ts` to add the LoL⁺ throughput measurement module. We also added the sliding window bandwidth estimator (`sw-bandwidth-estimator.ts`) and harmonic mean bandwidth estimator (`sw-bandwidth-estimator.ts`) that are used by Stallion and Llama, respectively. We note that we added a logging class to hls.js in order to simplify collection of performance metrics.

4 EXPERIMENTAL EVALUATION

4.1 Implementation and Demo

We integrated different LLL ABR schemes in the JavaScript-based hls.js player (v1.0.0) [18]. Our source code [3] and a demonstrator [4] are available online. The implementation consists of approx. 2,350 lines of new or modified code (Table 1).

4.2 Methodology and Evaluation Setup

4.2.1 Video and Encoding Parameters. We used Apple’s HLS Tools [1], namely `tsrecompressor`, `mediastreamsegmenter`, and `ll-hls-origin-example.go`, to generate LL-HLS streams conforming to the LL-HLS specification. The `tsrecompressor` uses the *Big Bug Bunny* animation video as input to produce three HLS streams encoded using the following ABR ladder: {480p@0.5Mbps, 720p@2Mbps, 1080p@4Mbps}, and then it pushes them as MPEG-2 transport streams to local UDP ports. On the other hand, three instances of `mediastreamsegmenter` are created and listen to these ports. Each instance is created for one encoded HLS stream and is responsible to (i) chop the encoded stream into segments and parts with a duration of six seconds and one second, respectively, (ii) package each part, and (iii) list the set of segments with their information (including parts) in a sub-playlist. The resulting sub-playlists of the corresponding bitrate levels are then combined in a master playlist. Finally, `ll-hls-origin-example.go` acts as an HLS origin which is used to push LL-HLS streams to the player.

The live session duration is set according to the duration of the network profile (600 seconds).

4.2.2 Network Profiles. To emulate various real-world network conditions, we used three LTE-based network profiles to shape the bandwidth between the server and the player. These profiles (Avg: x Mbps, std: x Mbps), including BelgiumCAR (4.3, 2.5), NorwayMETRO (1.5, 0.5), and NyuBUS (9.6, 5.3), are extracted from three datasets and can be found in [7]. We fixed the packet delay to 50 ms and loss rate to 0.08%.

4.2.3 ABR Algorithms and Performance Metrics. We compare the performance of five ABR schemes (LoL⁺, L2A, Llama, Stallion, and Default HLS) in terms of QoE and its following metrics:

- Avg. BR: Average bitrate across the live session (Mbps).
- Avg. RD: Average rebuffering duration across the live session (s).
- Avg. RC: Average rebuffering count across the live session.
- Avg. SC: Average bitrate switching count across the live session.
- Avg. PS: Average playback speed across the live session (s).
- Avg. LL: Average end-to-end latency across the live session (s).

Default HLS (D-HLS) is a hybrid ABR scheme that combines the buffer level and measured throughput to perform ABR decisions. In a nutshell, D-HLS selects the bitrate for the next segment based on the latest throughput if the buffer is safe (more than two segments are buffered). Otherwise, if the buffer is endangered and the estimated throughput shows that there might be a risk of rebuffering, it will downshift and select the lowest bitrate level.

4.2.4 Setup. To build an end-to-end LL-HLS system, we used a MacBook Pro running macOS Catalina with 6-Core Intel i7 processor, 16 GB memory, Intel UHD Graphics 630, and two virtual machines (VMs). The first VM was used to run the hls.js player in the Google Chrome browser (v88). The second VM executed Apple’s HLS Tools to produce the HLS stream with LLL mode (encoding and packaging) to feed into the origin server. To emulate a realistic network, we used *tcNetEm* at the server VM to throttle the total bandwidth available to the clients according to the network profiles. We performed the experiments with two fixed target latency values of 1.5 and 3 seconds in `latency-controller.ts`.

4.3 Results and Analysis

For each ABR scheme, the experiments were performed five times using the same network profile and we took the average values as results. Tables 2 and 3 summarize the QoE metrics average results for each ABR for the considered network profiles.

As shown in Table 2, LoL⁺ achieved the best results in terms of Avg. BR, Avg. RC, and Avg. RD with an average improvement of 49.8% (209.1%), 59.2% (52.4%), and 79.3% (84.1%), respectively, compared to all other ABR schemes for both target latency configurations (1.5 and 3 seconds). This LoL⁺ result comes at the price of having higher Avg. SC which exactly matches its design initiative to react to the rebuffering more actively. Another

Table 2: Average results of ABR schemes on BelgiumCAR.

ABR	Avg. BR	Avg. RC	Avg. RD	Avg. SC	Avg. PS	Avg. LL
Target Latency: 1.5 seconds						
LoL ⁺	0.79	09	1.69	13	0.99	1.55
D-HLS	0.50	23	6.16	3	1.02	1.58
L2A	0.63	23	11.54	6	1.03	1.62
Llama	0.50	24	9.46	0	1.02	1.60
Stallion	0.50	19	7.45	4	1.02	1.60
Target Latency: 3 seconds						
LoL ⁺	2.18	03	0.19	17	0.97	2.91
D-HLS	0.51	08	2.60	3	1.01	2.96
L2A	0.92	05	1.62	5	1.01	3.06
Llama	0.50	06	0.77	0	1.01	3.08
Stallion	1.60	07	0.96	15	1.01	2.98

observation is that L2A suffered from long rebufferings especially for the 1.5 second target latency (with an Avg. RD of 11.54 seconds) due to inaccurate ABR decisions when the bandwidth is changing fast. This is in contrast to Stallion and LoL⁺ which react more quickly to any sudden change in the bandwidth. D-HLS achieves a comparable average result in general. In our tests, Llama did not switch the bitrate during the whole live session, which raised doubts about its reported performance. It will require further investigation and evaluation to test its function under more network conditions. We note that in all network profiles (NorwayMetro and NyuBUS are not shown), all the ABR schemes achieve consistent Avg. LL within the target latency with a slight increase in some cases. As seen in Table 2, different ABR algorithms may perform differently based on which metrics they tend to prioritize or sacrifice, which makes comparisons between these ABR schemes and drawing conclusions difficult tasks. One way to compare them is to find the best and worst performing scheme in each QoE metric and quantify the relative performance of the other schemes against these boundaries or targets so that a system implementer may choose a scheme based on his/her own priorities and preferences. Here, we followed this strategy for the result analysis. Therefore, if a higher bitrate is preferred, then LoL⁺, Stallion or L2A for the target latency of 3 seconds are the best fit. Otherwise, if better video stability, with fewer and shorter rebuffering events, are preferable, then Llama or D-HLS are good alternatives.

To compare the performance of the considered ABR schemes with LL-HLS versus LL-DASH, we replicate the same experiments and setup (refer to Section 4.2) using the dash.js reference player [9]. We use the target latency of 3 seconds and the LoL⁺ throughput measurement module in all ABR schemes for accurate bandwidth estimation. The average results for different network profiles are summarized in Table 3. We can extract four essential observations from these results: (i) The performance of the ABR schemes is very similar in LL-DASH and LL-HLS, mainly for Avg. BR and also Avg. LL, which remains within the target value. (ii) The hls.js player produces 6× more requests than dash.js. (iii) Llama prefers to select lower bitrate levels and never up-shifts to the highest bitrate level for different network profiles. (iv) The playback speed and bitrate decisions should be considered jointly to further reduce Avg. RC, Avg. RD, and Avg. LL. Based on (i–iv), we can conclude that both LL-DASH and LL-HLS are very promising technologies for LLL streaming. However, we believe that there is certainly room for more work to be performed to optimize the players and their modules, including ABR schemes, playback speed control, and throughput estimators, for large-scale deployments. Additionally, we need a new standard QoE model that can support LLL streaming.

Table 3: Average results of ABR schemes using LL-DASH.

ABR	Avg. BR	Avg. RC	Avg. RD	Avg. SC	Avg. PS	Avg. LL
BelgiumCAR						
LoL ⁺	2.20	3	0.23	15	1.02	2.98
L2A	1.81	7	3.34	6	1.02	3.02
Llama	0.65	6	0.24	4	1.05	3.04
Stallion	1.62	4	0.64	15	1.05	3.03
NorwayMETRO						
LoL ⁺	0.71	10	2.35	11	1.05	3.02
L2A	0.59	8	1.11	6	1.02	3.03
Llama	0.50	4	0.93	1	1.02	3.02
Stallion	0.73	19	3.06	7	1.05	3.09
NyuBUS						
LoL ⁺	3.56	10	0.97	15	0.95	2.95
L2A	1.38	2	0.99	7	1.01	3.01
Llama	0.91	3	0.55	4	1.00	3.01
Stallion	2.86	3	0.63	11	1.01	3.00

5 CONCLUSIONS

Recently, new ABR solutions (LoL⁺, L2A, Stallion, and Llama) were proposed to support LL-DASH, with integration into the dash.js reference player. However, their performance with LL-HLS had not been studied yet. In this paper, we first re-implement and integrate these solutions in the hls.js video player to support LL-HLS. The source code is available [3] and a demonstration is given [4]. We performed a set of experiments to demonstrate the feasibility and performance of these solutions under multiple network conditions. Our results show the capabilities of these ABR solutions with LL-HLS. We found that the performance of these ABR schemes over LL-HLS is similar to their deployment over LL-DASH.

REFERENCES

- [1] Apple's HTTP Live Streaming (HLS) Tools. [Online] Available: <https://tinyurl.com/c53j6una>. Accessed on Oct. 24, 2021.
- [2] CTA-5004: Web Application Video Ecosystem–Common Media Client Data. [Online] Available: <https://cdn.cta.tech/cta/media/media/resources/standards/pdfs/cta-5004-final.pdf>. Accessed on Feb. 20, 2021.
- [3] Our LL-HLS hls.js Code. [Online] Available: <https://github.com/llhls-ABRdemo/LLHLS-ABRs>. Accessed on Jan. 28, 2022.
- [4] Our LL-HLS hls.js Demo. [Online] Available: <https://llhls-abrdemo.github.io/demo/index.html>. Accessed on Jan. 28, 2022.
- [5] Apple. Protocol Extension for Low-Latency HLS. [Online] Available: <https://tinyurl.com/chzva5a>, 2019. Online; accessed on Jan. 22, 2022.
- [6] A. Bentaleb, M. N. Akcay, M. Lim, A. C. Begen, and R. Zimmermann. Catching the Moment with LoL+ in Twitch-Like Low-Latency Live Streaming Platforms. *IEEE TMM*, pages 1–1, 2021.
- [7] A. Bentaleb and *et al.* LoL⁺ over dash.js. [Online] Available: <https://github.com/NUSstreaming/LoL-plus/>, 2021. Online; accessed on Jan. 04, 2022.
- [8] A. Bentaleb, C. Timmerer, A. C. Begen, and R. Zimmermann. Bandwidth Prediction in Low-Latency Chunked Streaming. In *ACM NOSSDAV*, 2019.
- [9] DASH-IF. DASH Reference Player (dash.js). [Online] Available: <https://reference.dashif.org/dash.js/>, 2021. Online; accessed on Jan. 22, 2022.
- [10] DASH-IF and DVB. Low-latency Modes for DASH. [Online] Available: <https://dashif.org/news/low-latency-dash/>, 2019. Online; accessed on Jan. 22, 2022.
- [11] C. Gutterman, B. Fridman, T. Gilliland, Y. Hu, and G. Zussman. Stallion: Video adaptation algorithm for low-latency video streaming. In *ACM MMSys*, MMSys '20, New York, NY, USA, 2020. Association for Computing Machinery.
- [12] ISO/IEC. ISO/IEC 23000-19:2020 Information technology – Multimedia application format (MPEG-A) – Part 19: Common media application format (CMAF) for segmented media. [Online] Available: <https://www.iso.org/standard/79106.html>, 2020. Online; accessed on Feb. 04, 2022.
- [13] ISO/IEC. 2019. Information technology – dynamic adaptive streaming over http (dash) – part 1: Media presentation description and segment formats. International standard 23009-1:2019, International Organization for Standardization, Dec. 2019.
- [14] T. Karagioules, R. Mekuria, D. Griffioen, and A. Wagenaar. Online learning for low-latency adaptive streaming. In *ACM MMSys*, MMSys '20, New York, NY, USA, 2020. Association for Computing Machinery.
- [15] T. Kohonen. The SOM. *Proceedings of the IEEE*, 78(9):1464–1480, 1990.
- [16] T. Lyko, M. Broadbent, N. Race, M. Nilsson, P. Farrow, and S. Appleby. Llama – low latency adaptive media algorithm. In *2020 IEEE ISM*, 2020.
- [17] R. Pantos and W. May. HTTP Live Streaming. RFC 8216, Aug. 2017.
- [18] R. Walch *et al.* HTTP Live Streaming (HLS). [Online] Available: <https://github.com/video-dev/hls.js/>, 2021. Online; accessed on Jan. 04, 2022.